



UNIVERSITY OF MINNESOTA

Driven to Discover®

Resources and the File System

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

This course content is offered under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license.



Today

- The file system: paths, file I/O, and asynchronous streaming
- Offline resource management and the tool chain
- The asset conditioning pipeline
- Runtime resource management
- Resource lifetimes and memory allocation strategies
- Composite resources and cross-references
- Post-load initialization

Why Resources Matter

- Games are **multimedia experiences**: textures, meshes, animations, audio, collision data, game world layouts, and more
- A game engine must load and manage a wide variety of media
- Memory is usually scarce, so the engine must ensure **only one copy** of each resource exists in memory at any given time
- Most engines employ a **resource manager** (a.k.a. asset manager or media manager) to handle this

Why Wrap the File System?

- Every resource manager depends heavily on the **file system**
- Game engines often wrap the native file system API for two reasons:
 1. **Cross-platform support**: Shield the engine from OS-level differences
 2. **Extended functionality**: The OS may not provide everything the engine needs (e.g., file streaming)
- Console engines also need access to varied media: memory sticks, hard drives, DVD-ROM, Blu-ray, network file systems (Xbox Live, PSN)

File System API Overview

- A game engine's file system API typically provides:
 - Manipulating **file names and paths**
 - **Opening, closing, reading, and writing** individual files
 - **Scanning** the contents of a directory
 - Handling **asynchronous file I/O** requests (for streaming)

File Names and Paths

- A **path** is a string describing a file or directory location in the file system hierarchy
- General structure:
 - `volume/dir1/dir2/ ... /dirN/file-name`
- Consists of an optional **volume specifier**, followed by **path components** separated by a reserved separator character (`/` or `\`)
- The last component is the **file name** (if the path refers to a file) or the **target directory**

Differences across Operating Systems

- **Path separators:** UNIX uses `/`, DOS/older Windows uses `\`, recent Windows accepts both
- **Case sensitivity:** UNIX is case-sensitive; Windows is case-insensitive
 - Can cause cross-platform asset problems
- **Volumes:** UNIX has a single monolithic hierarchy (no volume specifiers); Windows uses drive letters (`C:`) and UNC paths (`\\computer\share`)
- **File name lengths:** DOS limited to 8.3; modern Windows allows long names with dots
- **Reserved characters:** Each OS disallows certain characters in paths

More OS Differences

- Both UNIX and Windows support a **current working directory (CWD)**
 - Set via `cd` command on both; queried via `pwd` on UNIX
- Windows has a **separate CWD per volume** and a **current working volume**
- Consoles use predefined **path prefixes** for volumes
 - PS3: `/dev_bdvd/` for Blu-ray, `/dev_hddx/` for hard disks
 - PS3 dev kit: `/app_home/` maps to a user-defined host path

Absolute and Relative Paths

- **Absolute path:** Specified relative to the root directory
 - Starts with a path separator (/ or \)
 - Examples: `C:\Windows\System32`, `/usr/local/bin/grep`
- **Relative path:** Specified relative to some other directory
 - No leading path separator
 - Examples: `System32` (relative to CWD `\Windows`), `bin/grep` (relative to CWD `/usr/local`)
- On Windows, both absolute and relative paths may have an optional **volume specifier**

Search Paths

- A **search path** is a string containing a **list of paths** separated by a special character (colon or semicolon)
- Searched when looking for a file (e.g., the OS searches the `PATH` environment variable for executables)
- Some engines use search paths to locate resources
 - OGRE uses a `resources.cfg` file listing directories and ZIP archives to search
- Searching at runtime is **slow**; if asset paths can be known ahead of time, searching can be avoided entirely

Path APIs

- Paths are more complex than simple strings: isolating directories, filenames, extensions, converting between absolute and relative
- Windows provides `shlwapi.dll` for path manipulation
- Sony provides similar APIs for PlayStation platforms
- Cross-platform engines typically implement a **stripped-down, custom path API**
 - Works on every target OS and meets the engine's particular needs
 - Can be a thin wrapper around native APIs or written from scratch

Basic File I/O

- The C standard library provides **two APIs** for file I/O:
 - **Buffered (stream I/O)**: The API manages input/output data buffers for you
 - **Unbuffered (low-level I/O)**: The programmer manages data buffers manually

Operation	Buffered	Unbuffered
Open	<code>fopen()</code>	<code>open()</code>
Close	<code>fclose()</code>	<code>close()</code>
Read	<code>fread()</code>	<code>read()</code>
Write	<code>fwrite()</code>	<code>write()</code>
Seek	<code>fseek()</code>	<code>seek()</code>
Tell offset	<code>ftell()</code>	<code>tell()</code>

Low-Level Native APIs

- On UNIX, the STL's unbuffered I/O routines **are** native OS calls
- On Windows, they are wrappers around an even lower-level API:
 - `CreateFile()`, `ReadFile()`, `WriteFile()`, `CloseFile()`
- Native APIs expose full details of the file system
 - (e.g., file security attributes)
- Some teams manage their own buffers for performance
 - Red Alert 3: Accumulated log output into a memory buffer, flushed to disk only when full, moved the flush to a **separate thread** to avoid stalling the game loop

To Wrap or Not to Wrap

- Many game engines wrap the file I/O API in **custom I/O functions**
- Three advantages to wrapping:
 1. **Consistent behavior** across all target platforms, even when native libraries are inconsistent or buggy
 2. **Simplified API** with only the functions the engine actually needs
 3. **Extended functionality**: Handle files on hard disk, DVD-ROM, Blu-ray, network (Xbox Live, PSN), memory sticks, and other removable media through a single interface

Synchronous File I/O

- Both C standard library file I/O APIs are **synchronous**
- The program making the I/O request must **wait** until data transfer is complete
- The calling function **blocks** until all data has been read or written

```
bool syncReadFile(const char* filePath,
                  U8* buffer,
                  size_t bufferSize,
                  size_t& rBytesRead)
{
    FILE* handle = fopen(filePath, "rb");
    if (handle)
    {
        // BLOCK here until all data has been read.
        size_t bytesRead = fread(buffer, 1,
                                bufferSize, handle);
        int err = ferror(handle); // get error if any
        fclose(handle);
        if (0 == err)
        {
            rBytesRead = bytesRead;
            return true;
        }
    }
    rBytesRead = 0;
    return false;
}
```

Asynchronous File I/O

- Many games provide **seamless, load-screen-free** experiences by **streaming** upcoming level data
- Audio and textures are the most commonly streamed data types, but any type of data can be streamed
- Requires an **asynchronous** file I/O library
- Some OSes provide async I/O out of the box:
 - Windows CLR: `System.IO.BeginRead()`, `System.IO.BeginWrite()`
 - PlayStation: the `fios` API

Asynchronous File I/O: Priorities and Deadlines

- File I/O is a **real-time system** subject to deadlines,
- Async I/O operations often have **varying priorities**
 - Streaming audio is higher priority than loading a texture or level chunk
- Async I/O systems must be able to **suspend lower-priority requests** so higher-priority requests meet their deadlines
- Some APIs allow setting **deadlines** on requests, and specifying behavior when a deadline is missed (cancel, notify, keep trying)

How Asynchronous File I/O Works

- I/O requests handled in a **separate thread**
- The main thread places requests on a **queue** and returns immediately
- The I/O thread picks up requests and handles them sequentially using **blocking I/O** (`read( )` or `fread( )`)
- When a request completes, a **callback** provided by the main thread is called
- If the main thread wants to wait, a **semaphore** is used
 - Each request has an associated semaphore
 - The main thread sleeps on the semaphore until the I/O thread signals completion

The Resource Manager

The Resource Manager

- Every game is built from a wide variety of **resources** (assets/media)
 - Meshes, materials, textures, shaders, animations, audio clips, level layouts, collision primitives, physics parameters, etc.
- Every game engine has a **resource manager** of some kind
- Two distinct but integrated components:
 1. **Offline**: Manages the tool chain that creates and transforms assets into their engine-ready form
 2. **Runtime**: Loads resources into memory, manages lifetimes, and unloads when no longer needed

Offline Resource Management

- In some engines, the resource manager is a **cleanly designed, unified, centralized subsystem**
- In others, it's spread across **disparate subsystems** written by different people at different times
- Regardless of implementation, the resource manager takes on well-understood responsibilities

Revision Control for Assets

- Small projects can manage assets as **loose files on a shared drive**; this doesn't scale
- Many teams use **source code revision control** (e.g., Perforce) to manage resources
 - Custom wrappers may help flatten the learning curve for artists
- **Data size** is a major challenge
 - Art files are much larger than source code
 - Copying everything locally can be slow and wasteful
- Solutions:
 - Commercial tools like **Alienbrain** designed for large data
 - Only sync files the user actually needs
 - Naughty Dog: Used **UNIX symbolic links** to avoid copying; local copies only for checked-out files

The Resource Database

- Most assets pass through an **asset conditioning pipeline (ACP)** before being used by the engine
- Each resource has **metadata** describing how it should be processed
 - Compression type for textures, frame ranges for animations, which mesh maps to which character, etc.
- This metadata is stored in a **resource database**
 - Blind data embedded in Maya files
 - Companion text files or XML files
 - True relational databases (MySQL, Microsoft Access)
 - Custom GUIs wrapping the database

Resource Database Requirements

- A resource database must provide:
 - Handling **multiple resource types** consistently
 - **Creating, deleting, inspecting**, and **modifying** resources
 - **Moving** resource source files between locations
 - **Cross-referencing** between resources (e.g., material used by a mesh)
 - Maintaining **referential integrity** of cross-references
 - A **revision history** with a log of who changed what and why
 - **Searching and querying** (e.g., find which levels use a particular animation)
- A well-designed resource database can make the difference between shipping a hit game and a project that never ships

Case Study: Unreal Engine

- Database managed by **UnrealEd**, an integrated tool for resource management, asset creation, and level layout
- Key benefits:
 - UnrealEd is **part of the game engine itself**, so assets can be previewed exactly as they appear in-game
 - **One-stop shopping**: The Generic Browser provides a single unified interface for all resource types
 - Assets are **validated on import**, providing immediate feedback to creators
- Drawbacks:
 - All data stored in large **binary package files** that are hard to merge in version control
 - Only one user can lock a package at a time, creating contention
 - Referential integrity is good but not perfect (dummy remapping objects accumulate)

Case Study: Naughty Dog

- Originally used **MySQL** for resource metadata; later moved to **XML files** managed under Perforce
- **Builder** GUI: Tree view of all resources + property editor
 - Supports actors, levels, meshes, skeletons, animations
 - Animations grouped into "bundles" for batch management
- Asset conditioning: Command-line tools (`ba name-of-actor` , `bl name-of-level`)
 - Tools query the database to determine how to export, process, and package assets
- Benefits: Granular resources, obvious mapping to source files, easy to build
- Drawbacks: No visualization tools outside the game itself, tools not fully integrated

Case Study: OGRE

- OGRE has a well-designed **runtime** resource manager
- Simple, consistent interface for loading any resource type
- Designed for **extensibility**
 - Easy to implement a manager for a new asset type
- Drawback:
 - Runtime-only solution
 - No offline resource database
 - Exporters must be run manually; metadata entered by the user each time

The Asset Conditioning Pipeline

- Resource data created in digital content creation (DCC) tools are usually not suitable for direct engine consumption
- The **asset conditioning pipeline (ACP)** processes source assets through three stages:
 1. **Exporters**: Get data out of the DCC's native format into an intermediate format (custom plug-ins for Maya, Photoshop, etc.)
 2. **Resource compilers**: Massage raw exported data to make it game-ready (e.g., triangle strip reordering, texture compression, arc-length computation)
 3. **Resource linkers**: Combine multiple resource files into a single package (similar to linking object files into an executable)

Resource Dependencies and Build Rules

- Game assets have **interdependencies**, much like source code
 - A mesh refers to materials, which refer to textures
- Dependencies affect:
 - The **order** in which assets must be processed
 - Which assets must be **rebuilt** when a source asset changes
- Dependencies also arise from **data format changes**
 - If the mesh format changes, all meshes may need to be reprocessed
 - Some engines include versioned formats and legacy-loading code; the downside is code bloat
- Every ACP needs **build rules** describing interdependencies, and a **build tool** (e.g., `make` or a custom system) to ensure correct, ordered rebuilds

Runtime Resource Management

- Handles loading, managing, and unloading assets at runtime
- Responsibilities include:
 - Ensuring **only one copy** of each unique resource is in memory
 - Managing resource **lifetimes**
 - Loading needed resources and **unloading** those no longer needed
 - Handling **composite resources**
 - a 3D model = mesh + materials + textures + skeleton + animations

Runtime Resource Manager Responsibilities (cont.)

- Maintaining **referential integrity** of cross-references within and between resources
- Managing **memory usage** and ensuring resources are stored in appropriate memory regions
- Performing **post-load initialization** on a per-resource-type basis ("logging in")
- Providing a **unified interface** for managing a wide variety of resource types (ideally extensible)
- Handling **streaming**

Resource File and Directory Organization

- Some engines (typically PC) manage each resource as a **separate loose file** within a directory tree
- Example directory structure for a hypothetical game:

```
SpaceEvaders/  
  Resources/  
    NPC/  
      Pirate/  
      Marine/  
    Player/  
    Weapons/  
      Pistol/  
      Rifle/  
      BFG/  
    Levels/  
      Level1/  
      Level2/  
    Objects/  
      Crate/  
      Barrel/
```


Packaging Resources

- Other engines package multiple resources into a **single file**
 - ZIP archives or proprietary formats
- Benefits of a single large file:
 - Reduced **seek times** (data organized sequentially on disk)
 - Eliminated per-file **open costs**
 - Faster overall **load times**
- SSDs don't suffer from seek time issues, but consoles have not always included SSDs as primary storage, so minimizing seeks has historically been important
- **Unreal Engine**: All resources stored in proprietary **package files** (.pak files)

Benefits of ZIP Archives

- OGRE supports both loose files and **ZIP archives**
- Benefits of the ZIP format:
 1. **Open format**: zlib and zziplib libraries are freely available
 2. **Virtual files retain paths**: A ZIP archive "looks like" a raw file system
 3. **Compression**: Reduces disk space and speeds up load times
 - Decompression can be faster than reading more data
 4. **Modularity**: Resources grouped and managed as a unit
 - Great for **localization**: Swap a single ZIP per language/region

Resource File Formats

- Each resource type may have a **different format**
- Some types use **standardized formats**:
 - Textures: TGA, PNG, TIFF, JPEG, BMP, or compressed formats like S3TC/DXTn
 - Meshes: OBJ, COLLADA
- Some SDKs offer **flexible generic formats**
 - Granny SDK: A single format for meshes, skeletons, and animations (repurposable for any data type)
- Many engines **roll their own formats** to:
 - Provide information not available in standard formats
 - Maximize offline processing, minimizing runtime load/processing time
 - Store data in a **binary layout** that matches the in-memory format exactly

Resource GUIDs

- Every resource needs a **globally unique identifier (GUID)**
- Most common: The resource's **file system path** (as a string or 32-bit hash)
 - Intuitive: Maps directly to a physical file
 - Guaranteed unique by the OS
- Some engines use a **128-bit hash code** assigned by a tool
- **Unreal Engine**: Packages contain internal folder hierarchies
 - GUID = package name + in-package path
 - Example: `Locust_Boomer.PhysicalMaterials.LocustBoomerLeather`

The Resource Registry

- To ensure only one copy of each resource is in memory, most resource managers maintain a **registry of loaded resources**
- Simplest implementation: A **dictionary** (key-value pairs)
 - Keys = resource GUIDs
 - Values = pointers to the resource in memory
- When a resource is requested:
 - Found in registry → return the existing pointer
 - Not found → load it (or return a failure code)
- **Problem with auto-loading on demand:**
 - Loading is slow (disk access, memory allocation, post-load init)
 - Can cause **frame rate hitches** or multi-second freezes

Two Approaches to Loading

1. Disallow loading during gameplay

- Load all resources for a level en masse while the player watches a **loading screen**

2. Asynchronous loading (streaming)

- While the player plays level X, the resources for level Y load in the background
- Provides a load-screen-free experience
- Considerably more difficult to implement

Resource Lifetime

- A resource's **lifetime** spans from when it is loaded into memory to when its memory is reclaimed
- Different resources have different lifetime requirements:
 - **Global resources:** Loaded at game start, resident for the entire game
 - Player character mesh, HUD textures/fonts, standard weapons
 - **Level-scoped resources:** In memory for the duration of a level, dumped when the player leaves
 - **Short-lived resources:** Loaded for a specific event (e.g., an in-game cinematic), then freed
 - **Streamed resources:** Loaded in small chunks as they play (e.g., music in 4 KiB buffers)

Resource Unloading with Reference Counting

- Many resources are **shared across levels**; we don't want to unload a resource only to reload it immediately for the next level
- Solution: **Reference counting**
 1. Traverse the new level's resource list; **increment** reference counts
 2. Traverse the outgoing level's resources; **decrement** reference counts
 3. Any resource whose count drops to **zero** is unloaded
 4. Any resource whose count went from zero to one is **loaded**

Resource Unloading with Reference Counting

- Example: Level X uses resources A, B, C; Level Y uses B, C, D, E
 - B and C are shared; they stay in memory across the level transition
 - A is unloaded; D and E are loaded

Event	A	B	C	D	E
Initial state	0	0	0	0	0
Level X counts incremented	1	1	1	0	0
Level X loads	(1)	(1)	(1)	0	0
Level X plays	1	1	1	0	0
Level Y counts incremented	1	2	2	1	1
Level X counts decremented	0	1	1	1	1
Level X unloads, level Y loads	(0)	1	1	(1)	(1)
Level Y plays	0	1	1	1	1

Memory Management for Resources

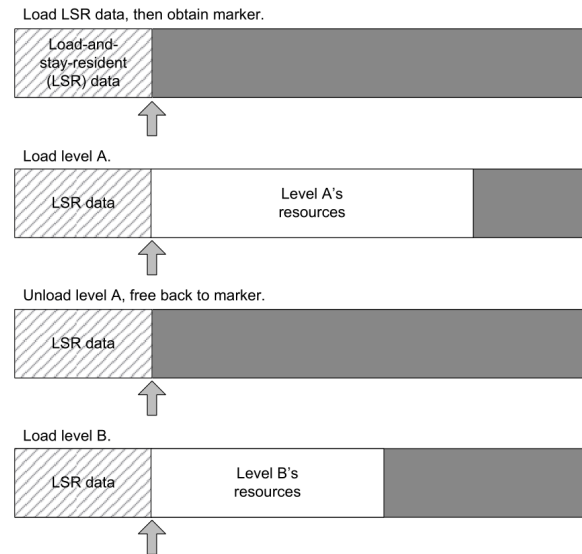
- Resource management is closely tied to **memory management**
- Different resource types may need to go to different memory regions:
 - **Video RAM** (or high-speed bus memory on PS4): Textures, vertex buffers, index buffers, shaders
 - **Main RAM**: Most other resources
 - **Different address ranges** within main RAM for different lifetime categories
- The memory allocation subsystem is often designed hand-in-hand with the resource manager
- Key challenge: Avoiding **memory fragmentation** as resources are loaded and unloaded

Heap-Based Resource Allocation

- Simply use a general-purpose **heap allocator** (`malloc()` or `new`)
- Works best on PCs with **virtual memory**
 - Physical memory fragments, but the OS maps noncontiguous physical pages into contiguous virtual addresses
- On consoles with limited RAM and no virtual memory, fragmentation becomes a real problem
- One option: **Defragment memory periodically**

Stack-Based Resource Allocation

- A **stack allocator** avoids fragmentation entirely
 - Memory allocated contiguously, freed in reverse order
- Works when:
 - The game is **linear and level-centric**
(loading screen → play → loading screen → play)
 - Each level fits in memory entirely
- Process:
 1. Load **global resources** first
 2. **Mark** the top of the stack
 3. Load the level's resources on top
 4. When the level ends, **reset the stack** to the marker, freeing all level resources at once

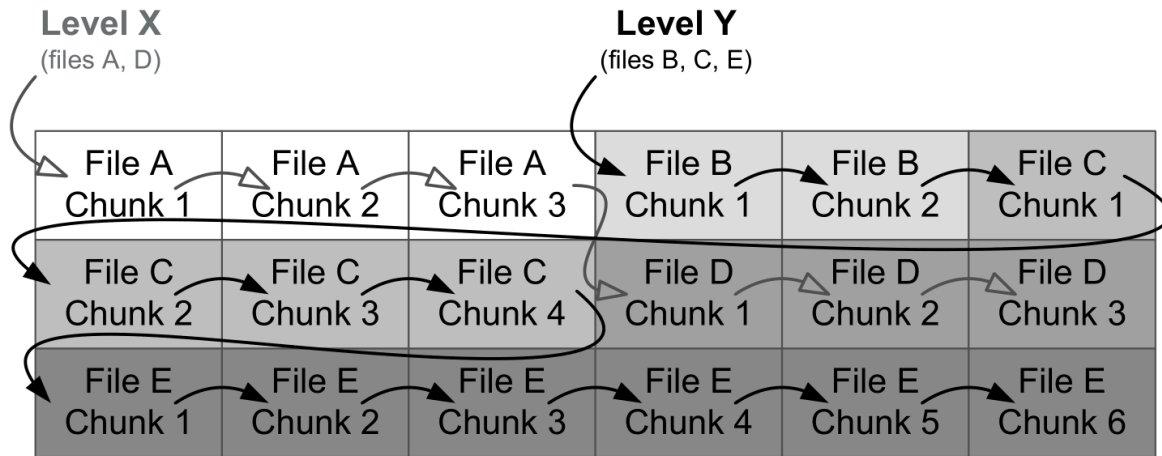


Pool-Based Resource Allocation

- Load resource data in **equally sized chunks**
- Chunks allocated using a **pool allocator**, freed without fragmentation
- Each chunk associated with a **particular game level** via a linked list
- When a level is unloaded, return its chunks to the free pool
- Typical chunk sizes: A few KiB to 1 MiB
 - Naughty Dog: 512 KiB on PS3, 1 MiB on PS4

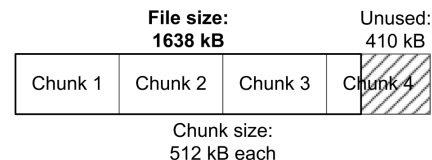
Pool-Based Resource Allocation

- Trade-off: **wasted space** in the last chunk of each file
- All resource data must be designed with "**chunkiness**" in mind
 - Avoid large contiguous data structures that span multiple chunks



Resource Chunk Allocators

- The unused portion of a resource's last chunk is wasted space
- A **resource chunk allocator** reclaims this unused memory for other allocations
- **Critical constraint:** When a chunk is freed, any allocations within it disappear
- Solution: Only allocate from a level's chunks when data share the **same lifetime**
 - Manage each level's chunks separately
 - Users must specify which level they are allocating for



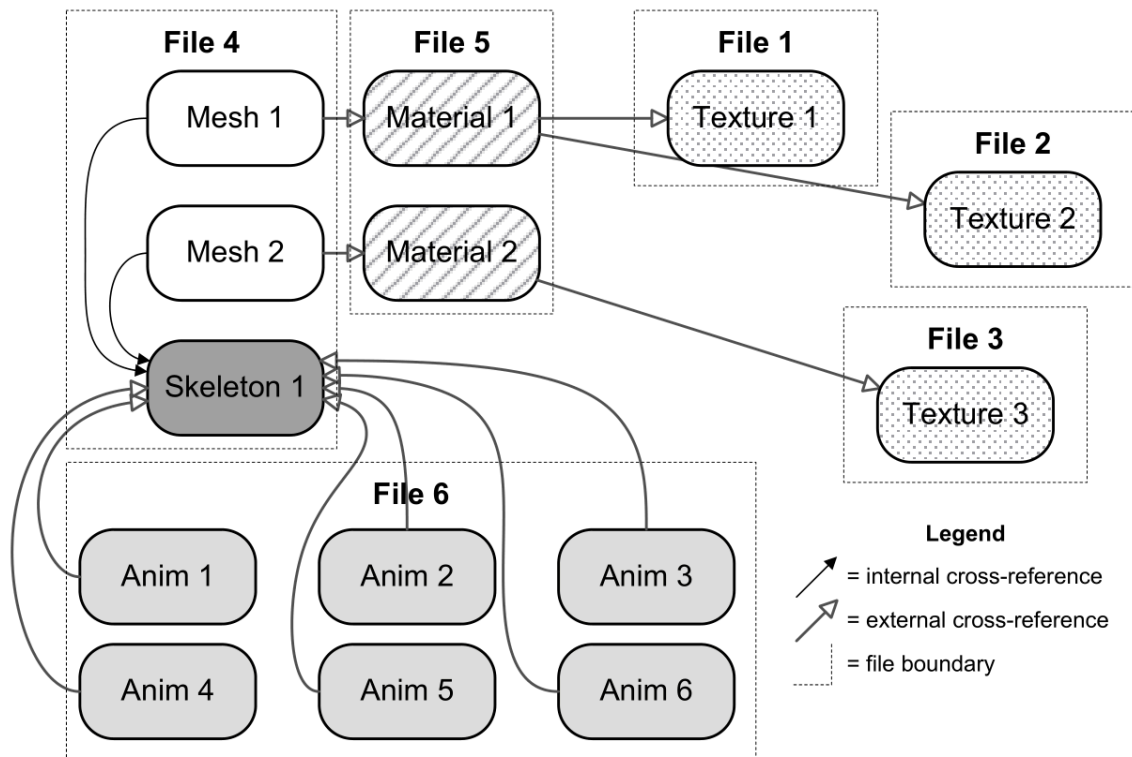
Sectioned Resource Files

- A resource file may contain between one and four **sections**, each divided into chunks
- Each section targets a different memory destination or purpose:
 - **Main RAM data**
 - **Video RAM data**
 - **Temporary data** needed only during loading, discarded afterward
 - **Debug data** loaded in debug mode, omitted in production builds

Composite Resources and Referential Integrity

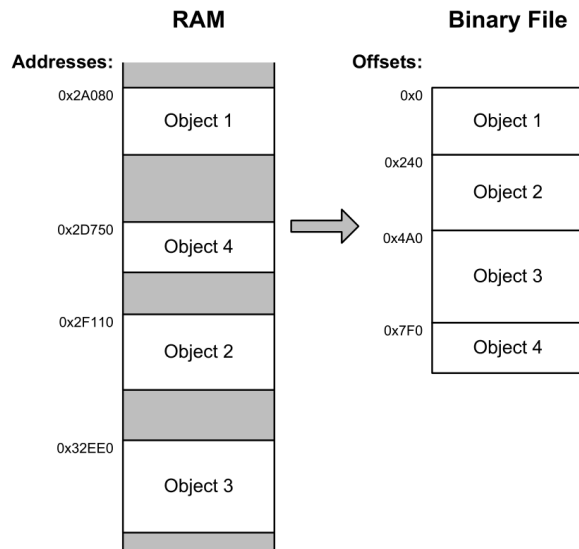
- A resource database consists of multiple files containing data objects that **reference each other**
- Example: A mesh references its material, which references textures
- Cross-references can be:
 - **Internal**: Between objects within a single file
 - **External**: Between objects in different files
- A **composite resource** is a self-sufficient cluster of interdependent resources
 - Example: A 3D model = meshes + skeleton + animations + materials + textures
- To fully load a composite resource, **all dependent resources** must be loaded as well

Resource Dependencies



The Problem with Cross-References

- In C++, cross-references are typically **pointers** or **references**
 - `Material* m_pMaterial`
 - `Material& m_material`
- Pointers are just **memory addresses** that lose meaning outside the running application
- Addresses can change between runs
- We **cannot use raw pointers** when storing data to disk



In-memory object images become contiguous when saved into a binary file.

Cross-References: GUIDs

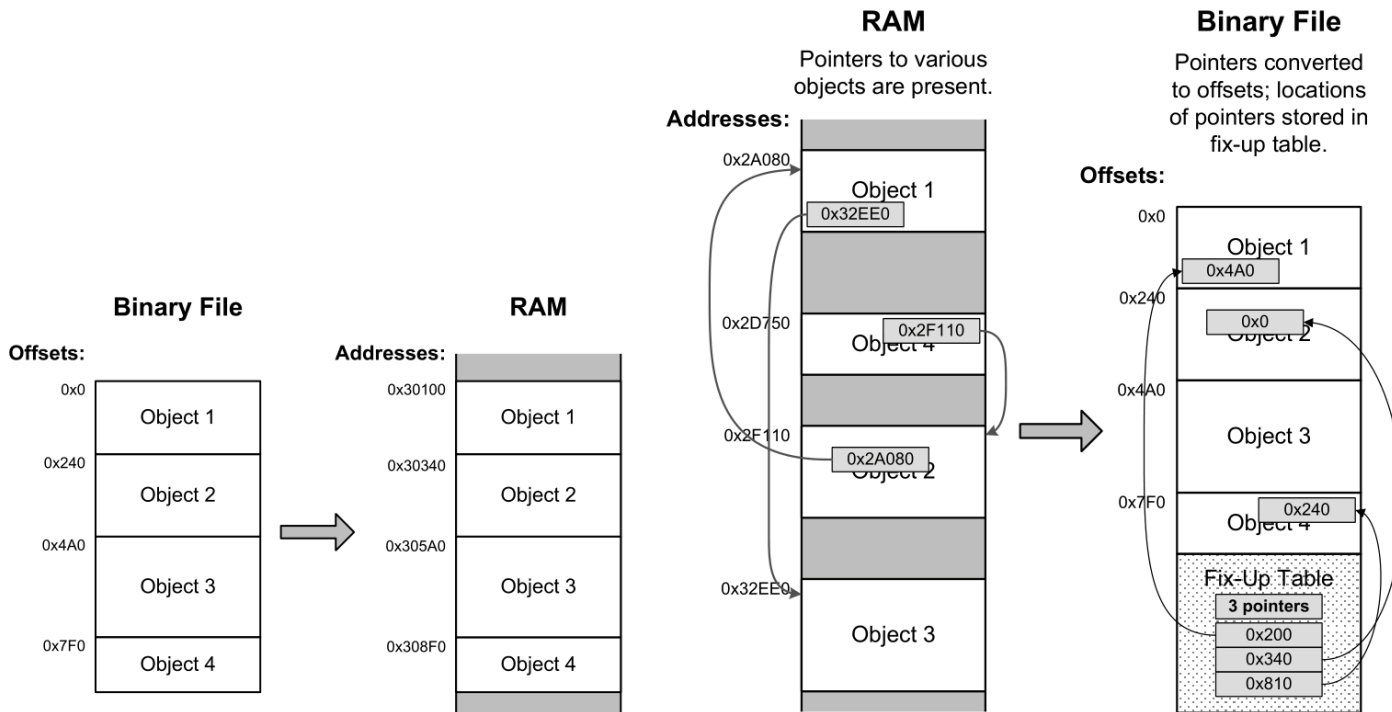
- Store each cross-reference as a **string or hash code** containing the referenced object's GUID
- At load time, the runtime resource manager maintains a **global resource look-up table**
- Process:
 1. Load each resource object into memory
 2. Add each object's pointer to the table, keyed by its GUID
 3. After all objects are loaded, make a pass over all objects to **convert GUIDs into pointers** using the look-up table

Cross-References: Pointer Fix-Up Tables

- When writing objects into a binary file, convert **pointers into file offsets**
- Objects are serialized into a **contiguous image** within the file
- Every pointer location is recorded in a **pointer fix-up table** stored alongside the data
- At load time:
 - The binary image is loaded into memory, preserving contiguous layout
 - Each offset is converted back to a pointer by **adding the base address** of the loaded image

```
U8* ConvertOffsetToPointer(U32 objectOffset, U8* pAddressOfFileImage)
{
    U8* pObject = pAddressOfFileImage + objectOffset;
    return pObject;
}
```

Cross-References: Pointer Fix-Up Tables



Storing C++ Objects as Binary Images

- When loading C++ objects from binary, **constructors** must still be called
- Two common solutions:
 1. **Restrict to POD types**: Only use plain old data structures (C structs, classes with no virtual functions and trivial constructors)
 2. **Constructor table**: Save offsets and types of non-POD objects; after loading, iterate the table and call constructors via **placement new**:

```
void* pObject = ConvertOffsetToPointer(objectOffset, pAddressOfFileImage);  
::new(pObject) ClassName; // placement new syntax
```

Handling External References

- Internal cross-references (within one file) can be resolved with pointer fix-ups after loading a single file
- **External cross-references** must also specify the **path to the resource file** containing the referenced object
- Loading a multi-file composite resource:
 1. Load one resource file
 2. Scan its cross-reference table; load any **externally referenced files** not yet loaded
 3. Add each loaded object's address to a **master look-up table**
 4. After all interdependent files are loaded, make a final pass to **fix up all pointers** using the master look-up table

Post-Load Initialization

- Ideally, resources are **fully prepared** by offline tools, ready to use the moment they are loaded
- In practice, many resource types need **post-load processing** ("logging in")
 - Most resource managers also support a **tear-down step** before freeing memory ("logging out")
- Two varieties of post-load initialization:

1. Unavoidable

- Example: On PC, mesh vertex/index data must be transferred from main RAM into **video RAM** via buffer creation, locking, copying, and unlocking

2. Avoidable but convenient:

- Example: Calculating arc lengths at runtime instead of modifying the tools to do it offline

Post-Load Initialization: Implementation

- Each resource type has unique post-load and tear-down needs
- Resource managers allow these steps to be **configurable per resource type**
- In C++, use **polymorphism**:
 - Each class handles initialization and tear-down via virtual functions (e.g., `Init()` and `Destroy()`)
 - Not done in constructors/destructors, because objects must be constructed first, then pointers fixed up, then initialization performed as a **separate step**
- Post-load init is tied to **memory allocation**:
 - New data may **augment** loaded data (e.g., computing additional metadata)
 - New data may **replace** loaded data (e.g., format conversion for backward compatibility)

Key Takeaways

- Game engines wrap the **file system** to ensure cross-platform consistency and provide extended features like streaming
- **Paths** are complex: OS-specific separators, case sensitivity, volume specifiers, and reserved characters all require careful handling
- **Synchronous I/O** blocks the caller; **asynchronous I/O** uses a separate thread with callbacks or deferred waits to keep the game running
- Async I/O requires **priority management** to ensure time-critical data meets their deadlines (e.g., audio)

Key Takeaways

- Every game engine has a **resource manager** with both offline and runtime components
- The offline side includes **revision control**, a **resource database**, and an **asset conditioning pipeline** (export → compile → link)
- Build rules and dependency tracking are critical to keeping assets consistent and avoiding wasted rebuild time
- Successful resource database designs (Unreal, Naughty Dog, OGRE) make different trade-offs but all address the same core needs

Key Takeaways

- The runtime resource manager ensures **one copy** per resource, manages **lifetimes**, and maintains **referential integrity**
- **Reference counting** prevents unnecessary unload/reload cycles when resources are shared across levels
- Memory allocation strategies (heap, stack, pool, chunk) each address **fragmentation** and lifetime management differently
- **Cross-references** are stored on disk as GUIDs or file offsets and resolved into pointers at load time via look-up tables or pointer fix-up tables
- **Post-load initialization** handles any processing that cannot (or has not yet been) moved to the offline tools



Questions?